

Software Lab08

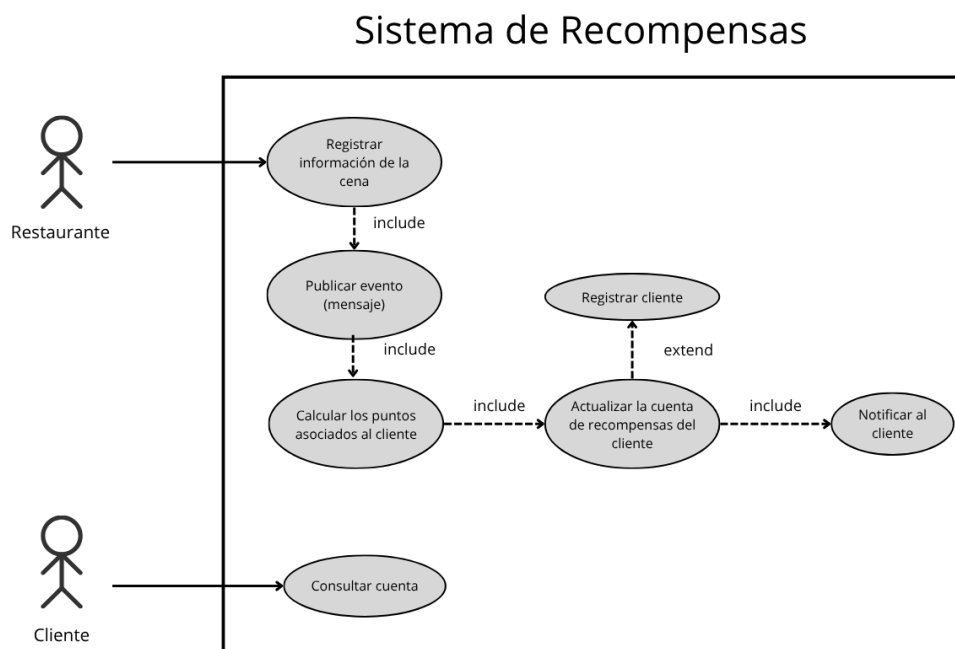
Repositorio: [GitHub – Software_Lab08](#) Análisis SonarQube: [Ver dashboard](#)

1. Introducción

El proyecto implementa un sistema distribuido orientado al proceso de registro de cenas, cálculo de recompensas y notificación al cliente. La solución combina servicios HTTP, mensajería asíncrona con RabbitMQ y persistencia en memoria para facilitar pruebas automatizadas y validación funcional.

2. Diagrama de caso de uso

La figura de caso de uso se presenta como referencia funcional del sistema.



En este flujo intervienen principalmente dos actores externos. El **Restaurante** registra la información de la cena realizada, mientras que el **Cliente** participa como beneficiario de las recompensas y como actor asociado a la consulta posterior de su cuenta. A partir de esa interacción, el sistema procesa internamente la transacción y publica un mensaje en RabbitMQ con el monto consumido, el número de tarjeta, el código del restaurante y la fecha y hora de la transacción.

Después, el microservicio de recompensas consume el mensaje, calcula los puntos y cashback asociados, y actualiza la cuenta del cliente. Si no existe una cuenta previa asociada al identificador de la tarjeta, el sistema crea automáticamente una

nueva cuenta para no interrumpir el procesamiento. Posteriormente, se genera un evento adicional para notificar al cliente por correo o por otro canal equivalente.

La decisión de crear automáticamente una nueva cuenta de recompensas cuando no existe una asociación previa responde a dos criterios:

1. El enunciado del laboratorio no especifica una restricción explícita que obligue a registrar al cliente antes de procesar una cena.
2. La implementación busca mantener el flujo operativo sin bloquear el cálculo de recompensas, permitiendo que el sistema continúe aun cuando el cliente no tenga un registro previo.

En ese sentido, el comportamiento automático funciona como una regla interna de consistencia del dominio y no como una acción manual del actor.

3. Arquitectura implementada

La arquitectura corresponde a un enfoque de **microservicios** con una organización interna de estilo **hexagonal / ports and adapters**. Cada servicio separa claramente el dominio de la infraestructura y expone una API o consumidor específico según su responsabilidad.

La solución se organiza en cuatro capas principales:

Capa	Responsabilidad	Ejemplos
Presentación	Exponer endpoints HTTP y puntos de arranque	main.py, api.py
Aplicación	Orquestrar casos de uso	application/use_cases.py
Dominio	Modelar entidades y contratos	domain/entities.py, domain/ports.py
Infraestructura	Implementar repositorios, publicadores y consumidores	infrastructure/...

Además, existe un módulo compartido con configuración, salud y mensajería base en `app/shared/`.

3.1 Justificación de la arquitectura observada

La implementación evidencia un grado alto de madurez arquitectónica por las siguientes razones:

- **Alta cohesión:** cada servicio agrupa una responsabilidad concreta. `customer_service` gestiona clientes, `dinner_service` registra cenas, `rewards_service` procesa recompensas y `notifications_service` emite notificaciones.
- **Bajo acoplamiento:** las dependencias entre capas se resuelven mediante puertos e interfaces abstractas. El dominio no conoce las clases concretas de infraestructura, y los casos de uso reciben repositorios, publishers o senders como dependencias inyectadas.
- **Modularidad:** el código está dividido por servicio y por capa, lo que permite modificar una parte sin impactar directamente a las demás. Además, los adaptadores en memoria y RabbitMQ pueden sustituirse sin cambiar la lógica central.
- **Escalabilidad:** la separación por servicios y el uso de colas de mensajería permite escalar de forma independiente los componentes. Al desacoplar la publicación del evento y su procesamiento, el sistema puede absorber picos de demanda y distribuir la carga según la necesidad operativa.
- **Arquitectura orientada a eventos:** el flujo principal no depende solo de llamadas síncronas. El registro de una cena genera un evento que viaja por RabbitMQ, luego otro servicio lo consume, procesa recompensas y publica un nuevo evento para notificación.

3.2 Justificación de RabbitMQ frente a otras opciones

RabbitMQ fue la opción más adecuada para esta implementación por la naturaleza del problema y por el alcance del laboratorio:

1. **Adecuación al problema:** el laboratorio necesita entregar mensajes confiables entre productores y consumidores. RabbitMQ encaja mejor que Kafka porque se ajusta a colas de trabajo y eventos discretos, sin introducir la complejidad de streaming o retención histórica prolongada.
2. **Simplicidad operativa y de pruebas:** para un entorno académico con pruebas automatizadas y ejecución local, RabbitMQ resulta más fácil de configurar, observar y validar. Permite trabajar con ACKs, colas y exchanges sin infraestructura adicional innecesaria.
3. **Comparación con ActiveMQ:** aunque ActiveMQ también cumpliría la función de mensajería, RabbitMQ ofrece una integración más natural con el patrón de productor-consumidor usado en esta implementación.

4. Patrón arquitectónico utilizado

El patrón predominante es **Arquitectura Hexagonal (Ports and Adapters)**, complementado con una **arquitectura orientada a eventos (EDA)**.

Elemento	Función
Ports	Interfaces abstractas del dominio (ports.py)
Adapters	Implementaciones concretas en memoria o RabbitMQ
Use cases	Reglas de negocio orquestadas desde la capa de aplicación
Mensajería	Comunicación asíncrona entre servicios

Este enfoque reduce el acoplamiento entre negocio e infraestructura y permite que los tests sustituyan fácilmente repositorios, publishers y consumidores por dobles de prueba.

5. Evidencia de pruebas automatizadas

La ejecución automatizada de pruebas confirma que la suite pasa correctamente.

```

===== test session starts =====
platform win32 -- Python 3.12.10, pytest-7.4.0, pluggy-1.6.0
rootdir: C:\Users\ERIKA\Documents\JHO GAM\Ciclo 5\Software\Labs\Software_Lab08
plugins: anyio-4.13.0, cov-4.1.0
collected 98 items

tests\e2e\test_notification_flow.py . [ 1%]
tests\e2e\test_reward_flow.py . [ 2%]
tests\integration\test_account_and_notification.py . [ 3%]
tests\integration\test_api.py . [ 4%]
tests\integration\test_customer.py . [ 5%]
tests\integration\test_dinner_integration.py ..... [ 12%]
tests\integration\test_notifications_integration.py ..... [ 21%]
tests\integration\test_rabbitmq_real.py . [ 22%]
tests\integration\test_rewards.py .. [ 24%]
tests\unit\customer_service\test_customer_unit.py ..... [ 35%]
tests\unit\dinner_service\test_dinner_unit.py ..... [ 43%]
tests\unit\dinner_service\test_rabbit_publisher.py .. [ 45%]
tests\unit\notifications_service\test_main.py . [ 46%]
tests\unit\notifications_service\test_notifications_unit.py ..... [ 62%]
tests\unit\notifications_service\test_rabbit_publisher.py .. [ 64%]
tests\unit\rwards_service\test_main.py . [ 65%]
tests\unit\rwards_service\test_rewards_unit.py ..... [ 83%]
tests\unit\shared\test_events_unit.py .. [ 85%]
tests\unit\shared\test_messaging_base_unit.py ... [ 88%]
tests\unit\shared\test_ports_contracts.py .... [ 92%]
tests\unit\shared\test_shared_unit.py ..... [ 100%]

===== 98 passed in 14.52s =====

```

Para ejecutar las pruebas automatizadas se puede seguir uno de estos dos estilos:

1. Activar previamente el entorno virtual y usar los comandos de manera directa.
2. Llamar al intérprete del entorno virtual con la ruta explícita cuando no se desea activar la sesión.

En este informe se usará el primer estilo. Comando de prueba:

```
pytest
```

6. Cobertura de código

El archivo `coverage.xml` reporta una cobertura total de **96,92 %** sobre **389 líneas válidas**, con **377 líneas cubiertas**. En la captura de cobertura se observa un resultado global redondeado de **97 %**.

```
----- coverage: platform win32, python 3.12.10-final-0 -----
```

Name	Stmts	Miss	Cover	Missing
app__init__.py	0	0	100%	
app\services\customer_service\application\use_cases.py	9	0	100%	
app\services\customer_service\domain\entities.py	6	0	100%	
app\services\customer_service\domain\ports.py	9	0	100%	
app\services\customer_service\infrastructure\repository\in_memory.py	9	0	100%	
app\services\customer_service\main.py	18	1	94%	27
app\services\customer_service\store.py	2	0	100%	
app\services\dinner_service\application\use_cases.py	8	0	100%	
app\services\customer_service\infrastructure\repository\in_memory.py	9	0	100%	
app\services\customer_service\main.py	18	1	94%	27
app\services\customer_service\store.py	2	0	100%	
app\services\dinner_service\application\use_cases.py	8	0	100%	
app\services\dinner_service\domain\entities.py	8	0	100%	
app\services\dinner_service\domain\ports.py	6	0	100%	
app\services\dinner_service\infrastructure\messaging\rabbit_publisher.py	20	0	100%	
app\services\dinner_service\main.py	30	2	93%	23, 49
app\services\notifications_service\application\use_cases.py	10	0	100%	
app\services\notifications_service\domain\entities.py	8	0	100%	
app\services\notifications_service\domain\ports.py	6	0	100%	
app\services\notifications_service\infrastructure\delivery\in_memory.py	7	0	100%	
app\services\notifications_service\infrastructure\messaging\consumer.py	13	0	100%	
app\services\notifications_service\infrastructure\messaging\publisher.py	13	0	100%	
app\services\notifications_service\main.py	10	1	90%	14
app\services\rewards_service\api.py	9	1	89%	13
app\services\rewards_service\application\use_cases.py	33	2	94%	13-14
app\services\rewards_service\domain\entities.py	8	0	100%	
app\services\rewards_service\domain\ports.py	9	0	100%	
app\services\rewards_service\infrastructure\messaging\consumer.py	17	0	100%	
app\services\rewards_service\infrastructure\repository\in_memory.py	9	0	100%	
app\services\rewards_service\main.py	11	1	91%	15
app\services\rewards_service\store.py	2	0	100%	
app\shared\config\settings.py	14	0	100%	
app\shared\health.py	13	0	100%	
app\shared\messaging\base.py	44	3	93%	35, 72, 76
app\shared\messaging\events.py	14	0	100%	
app\shared\messaging\rabbitmq.py	14	1	93%	29
TOTAL	389	12	97%	

Coverage XML written to file coverage.xml

Para generar nuevamente el reporte de cobertura se puede usar:

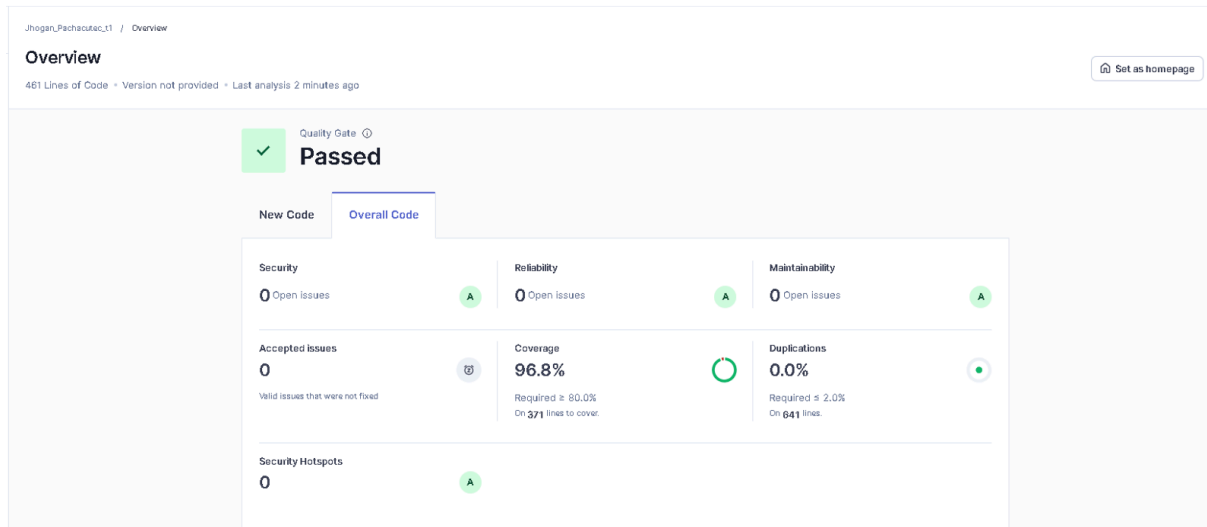
```
python -m pytest --cov=app --cov-report=term-missing
--cov-report=xml
```

7. Métricas de calidad en SonarQube

El análisis de SonarQube muestra un estado favorable del proyecto:

- **Quality Gate:** Passed
- **Security:** 0 issues abiertas
- **Reliability:** 0 issues abiertas
- **Maintainability:** 0 issues abiertas

- **Duplications:** 0,0 %
- **Coverage en SonarQube:** 96,8 %
- **Security Hotspots:** 0



8. Ejecución local del sistema

El proyecto fue trabajado con un entorno virtual de Python previamente creado. Antes de ejecutar los servicios o las pruebas, se recomienda activar ese entorno e instalar las dependencias del archivo `requirements.txt`:

```
pip install -r requirements.txt
```

Los comandos de ejecución de los componentes principales son:

```
python -m app.services.customer_service.main
python -m app.services.dinner_service.main
python -m app.services.notifications_service.main
python -m app.services.rewards_service.main
```